

임요섭 (신입)

Lim Yosup

2000. 06. 28

E-Mail : yoxup.lim@gmail.com

H.P : 010-9139-4102

Blog : <https://bdisappointed.tistory.com>

Portfolio : <https://xub2.github.io>



Introduction

팀의 WIKI가 되는 백엔드 개발자

데브 코스에서 Springboot / Java 기반 MSA 프로젝트를 경험하며 팀장으로서 최우수 프로젝트를 달성했습니다.

그 과정에서 새로운 도구보다 지금 가진 것으로 먼저 풀 수 있는가를 항상 먼저 고민했습니다.

그 고민을 꾸준히 문서로 남기면서, 현재는 AI 활용의 중요한 컨텍스트로도 활용하고 있습니다.

또한, 학부부터 현재까지 270편 이상의 기술 블로그를 운영하며 꾸준히 기록하는 습관을 이어오고 있습니다.

Project 1. 팀 프로젝트

원데이 클래스 예약 플랫폼 : 잡아 클래스 (Jaba Class)

프로젝트 개요 MSA 아키텍처 학습과 AI 기반 사용자별 클래스 추천 기능을 적용한 이커머스 프로젝트입니다.

기간 / 인원 2026.03 - 2026.05 / 6인

Github https://github.com/prgrms-be-adv-devcourse/beadv5_5_ChamomileChicken_BE

Technical Challenges

1. MD 기반 문서화 문화 구축과 Claude Code 컨텍스트 활용

MSA 7개 서비스를 팀원들이 나눠 개발하다 보니, 다른 팀원의 모듈을 파악하는 데 매번 시간이 소요됐습니다.

Claude Code를 도입했지만 세션이 초기화될 때마다 컨텍스트를 다시 설명해야 하는 비효율도 있었습니다.

이를 해결하기 위해 서비스별 설계 결정 / API 명세 / 장애 대응 흐름을 직접 MD로 정리하며 팀 내 문서화 문화를 만들었고, 작성한 MD를 Claude Code의 컨텍스트로 활용해 다른 팀원의 모듈도 빠르게 파악할 수 있는 개발 환경을 구축했습니다.

2. 서버 리소스 이슈 #1 — Elasticsearch 도입 결정 과정

원데이 클래스 예약 플랫폼 특성상 셀러명, 클래스 제목 기반의 검색 품질이 서비스 핵심이었습니다.

Elasticsearch(이하 ES)가 검색엔진으로 적합하다는 건 알고 있었지만, t3.large 위에서 7개 서비스에 Redis, Kafka, DB 까지 올리다 보니 추가 인프라 도입에 신중해야 했습니다.

PostgreSQL이 pg_trgm GIN 인덱스 확장을 제공하고 있어, ES 없이도 DB 레벨에서 해결할 수 있다고 판단해 먼저 적용했습니다.

- k6로 100만 건 데이터 / 최대 동시 100 VU 부하 테스트를 진행한 결과 GIN 적용 후 p95 1,879ms → 1,654ms로 12% 개선에 그쳤고, 극한 부하에서 실패 14건 발생.
- 검색 실패는 예약 기회 손실로 직결되는 서비스 특성상 읽기 요청이라도 실패 허용 불가.

수치를 기반으로 **ES 도입을 결정**했고, nori 형태소 분석기 적용으로 한국어 전문 검색을 지원하여 p95 1,294ms로 Full Scan 대비 31% 개선 및 실패율 0%를 달성했습니다.

3. 서버 리소스 이슈 #2 — 단일 ES 노드로 인한 SPOF 문제 해결

검색 로직 전체를 ES에 위임한 구조였기 때문에, ES 장애 발생 시 검색 API 전체가 오류로 이어지는 **단일 장애점 (SPOF) 문제**가 있었습니다. t3.large 리소스 제약으로 ES 노드를 추가로 띄울 수 없는 상황이었기에 인프라 확장 대신 애플리케이션 레벨에서 해결책을 찾아야 했습니다.

Resilience4j **Circuit Breaker**를 도입해 ES 장애 감지 시 즉시 PostgreSQL 직접 조회로 fallback 전환하도록 구성했습니다. 실제 장애 상황을 재현하기 위해 **ES 컨테이너를 강제 종료하는 카오스 테스트**를 설계하여 진행했고, CB 상태 전이와 fallback 동작을 검증했습니다.

- ES 장애 중 검색 API 실패율 0% 유지 (PostgreSQL fallback으로 응답 지속)
- ES 복구 후 CB 자동 전환 및 정상 동작 확인

4. 검색 기능 장애 대응 — Race Condition으로 인한 ES Dynamic Mapping 문제

서버 비용 절감을 위해 매일 EC2를 꺾다 켜는 운영 방식을 택했습니다. 재시작 시 Kafka에 소비되지 않은 메시지가 남아 있었고, consumer가 인덱스 초기화보다 먼저 실행되면서 nori 분석기가 빠진 **잘못된 매핑의 인덱스가 Dynamic Mapping으로 디스크에 고착**되는 문제가 발생했습니다. 이후 색인된 데이터들이 잘못된 매핑 위에 쌓이면서 검색 API 호출 시 한국어 형태소 분석이 되지 않는 문제(검색 기능 장애)로 이어졌습니다.

원인 파악 후 **@PostConstruct로 인덱스 초기화 순서를 보장**하고, 명시적 매핑을 직접 정의해 Dynamic Mapping을 차단함으로써 문제를 해결했습니다.

5. Admin 모듈 다중 도메인 DataSource 통합과 커넥션 풀 최적화

MSA 구조였지만 DB는 단일 PostgreSQL을 공유했고, 팀 규칙상 각 서비스는 서로의 테이블을 직접 참조하지 않는 것을 원칙으로 했습니다. 단, Admin 서비스는 예외적으로 5개 도메인 테이블에 동시 접근이 필요했습니다.

- 초기엔 도메인별 DataSource 4개로 분리되어 각각 HikariCP 기본값(10개)씩, **이론상 최대 40개 커넥션을 점유**하는 구조였습니다.
- **DataSource를 AdminDataSourceConfig 하나로 통합**하고 **TransactionManager도 1개로 정리**, 트래픽이 적은 Admin 특성을 고려해 maximum-pool-size: 2로 튜닝해 최대 **점유 커넥션을 40개 → 2개로 축소**했습니다.

Project 2. 개인 프로젝트

다니엘 청년부 행정 관리 웹 애플리케이션

프로젝트 개요	교회 청년부에서 출석 관리, 알림지 배포, 소그룹 편성 등 매주 반복되는 행정 업무를 직접 자동화하기 위해 만든 웹 서비스입니다. 실제 청년부 구성원 30 ~ 40명이 사용했으며, 매주 운영위원회 피드백을 반영하며 운영했습니다.
기간 / 인원	2025.12 - 2026.03
Github	https://github.com/xub2/youth

Technical Challenges

1. 출석 중복 등록 문제 — DB Unique 제약 기반 동시성 처리

운영위원들이 동시에 같은 인원의 출석을 등록하는 경우 데이터 중복이 발생하는 문제가 있었습니다. 락보다 구현이 단순하고 DB 레벨에서 항상 보장되는 **Unique 제약이 이 상황에 더 적합하다고 판단**해 적용했고, 중복 삽입 시도 자체를 DB 레벨에서 차단했습니다. 또한 예외 처리로 사용자에게 적절한 피드백을 반환하도록 구성했습니다.

- 애플리케이션 레벨이 아닌 DB 레벨에서 동시성 문제 근본 차단
- 데이터 중복률 0% 및 무결성 100% 달성

2. 전체 인원 일괄 갱신 성능 개선, JPA 벌크 연산 도입

매주 모임 시작 전 전체 인원의 출석 상태를 초기화해야 했습니다. 기존에는 인원 수(N)만큼 개별 UPDATE 쿼리가 실행되는 구조라, 인원이 늘어날수록 쿼리 횟수도 선형으로 증가하는 문제가 있었습니다.

- **JPA @Modifying**과 **JPQL 벌크 연산**으로 전환해, 인원 수만큼 반복되던 개별 UPDATE 쿼리를 전체 대상을 한 번에 갱신하는 단일 쿼리로 처리했습니다(N회 → 1회).

Education

2026.03 ~ 2026.05	프로그래머스 백엔드 단기심화 데브코스 (팀장)	<u>최우수 프로젝트(1위)</u>
2019.02 ~ 2026.03	평택대학교 ICT 융합학부 스마트콘텐츠전공 졸업	4.05 / 4.5

Certifications

2025.06	평택대학교 교내 학사 학위 논문 「REST API 개발을 통한 OWASP Top 10 보안 취약점 대응 방안 연구」 OWASP Top 10 기반 보안 취약점을 직접 재현하고, 대응 기법 적용 전후를 비교 분석한 실험 기반 연구
2024.12	SQL 개발자 (SQLD)